

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**ДЕМОН ЗА МАРШРУТИЗАЦИЯ ПО
СЪСТОЯНИЕ НА ВРЪЗКИТЕ И
ДИСКРЕТНО-СЪБИТИЕН СИМУЛАТОР ЗА
ИЗСЛЕДВАНЕ НА СХОДИМОСТТА В ТСП/Р
МРЕЖИ**

КУРСОВ ПРОЕКТ

по дисциплина „Мобилни и стационарни комуникационни мрежи“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Доц. д-р инж. Георги Цочев, Доц. д-р инж. Георги Найденов, Доц. д-р Петко Стоянов

София, **[TODO: година на предаване]**

СЪДЪРЖАНИЕ

УВОД	1
I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД .	3
II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА	5
2.1. Семейство на протокола	5
2.2. Модел на времето	5
2.3. Език и инструментариум за изграждане	5
2.4. Еталон за сверяване на коректността	6
2.5. Обхват спрямо учебното съдържание	6
III. ПРОЕКТИРАНЕ НА СИСТЕМАТА	7
3.1. Обща структура	7
3.2. Абстракция върху времето и входа и изхода	7
3.3. Протоколно ядро	8
3.4. Записване на събития за анализ	8
IV. ПРОГРАМНА РЕАЛИЗАЦИЯ	9
4.1. Организация на изходния текст	9
4.2. Автомат за състоянията на съседите	9
4.3. Разпространение на обявленията	9
4.4. Изчисляване и инсталиране на маршрути	9
4.5. Съвместимост с еталонната реализация	10
V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА	11
5.1. Измервани величини	11
5.2. Топологии и сценарии	11
5.3. Управлявани параметри	11
5.4. Условия за валидност	11
5.5. Форма на резултатите	12
VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	13
6.1. Проведени измервания	13

6.2. Време за сходимост	13
6.3. Преходни цикли	13
6.4. Сверяване с еталонната реализация	14
6.5. Обсъждане	14
ЗАКЛЮЧЕНИЕ	15
ЛИТЕРАТУРА	16
ПРИЛОЖЕНИЯ	17

УВОД

Маршрутизацията в мрежите по стека TCP/IP се описва в учебната литература като решена задача: всеки маршрутизатор поддържа представа за топологията, изчислява най-кратките пътища и попълва маршрутна таблица. Поведението, което определя качеството на мрежата на практика, обаче не е в установеното състояние, а в прехода между две установени състояния. Когато една връзка откаже, различните възли научават за отказа в различен момент и за краткия интервал, докато представите им се разминават, мрежата може да образува преходни цикли и да губи пакети по пътища, които изглеждат валидни за всеки възел поотделно. Този интервал се нарича време за сходимост и е основният предмет на настоящия курсов проект.

Причината преходът да се изучава рядко в лабораторни условия е практическа. Отказът на връзка в реална мрежа настъпва в момент, който не се повтаря, а таймерите на протокола се задвижват от истински часовник. Две последователни наблюдения на едно и също събитие дават различни числа и разликата между тях не може да бъде отдадена на промяна в протокола. За да бъде измерим, преходът трябва да бъде възпроизводим, а възпроизводимостта изисква времето да бъде управлявано от програмата, а не от часовника на машината.

Целта на проекта е да се изгради демон за маршрутизация по състояние на връзките от семейството OSPF заедно с дискретно-събитиеен симулатор, който изпълнява същия програмен код върху управлявано време, и с този инструмент да се измери времето за сходимост и образуването на преходни цикли при отказ на връзка.

За постигането на целта са формулирани следните **задачи**:

1. литературен преглед на принципите на маршрутизация и управление на потоците в мрежи по TCP/IP, поставен в контекста на дисциплината;
2. анализ на възможните решения по езика, модела на времето, транспортния слой и еталонната реализация, и обосновка на направения избор;
3. проектиране на демона така, че протоколната логика да не зависи от източника на време и от източника на пакети;
4. програмна реализация на откриване на съседи, разпространение на обявленията за състояние на връзките, изчисляване на най-кратките пътища и инсталиране на маршрути;

5. изграждане на методика за възпроизводимо измерване на сходимостта при отказ на връзка;
6. провеждане на измерванията и анализ на получените резултати.

Обхватът на реализацията е маршрутизиращият слой на стека TCP/IP. Безжичните, спътниковите и клетъчните мрежи, както и ATM, присъстват в раздел I като контекст, защото са част от учебното съдържание на дисциплината, но не влизат в реализацията. Причината е методологична, а не липса на интерес: тези среди въвеждат модел на канала със загуби и променлив капацитет, чиято достоверност сама по себе си е отделна изследователска задача, и включването им би направило измерената сходимост зависима от достоверността на модела на канала вместо от поведението на протокола.

Структура на изложението. Раздел I представя предметната област. Раздел II излага разгледаните алтернативи и обосновава избора. Раздел III описва проектирането, а раздел IV програмната реализация. Раздел V описва методиката за измерване, а раздел VI получените резултати. Заключение обобщава изводите и очертава посоките за по-нататъшна работа.

I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД

Дисциплината „Мобилни и стационарни комуникационни мрежи“ обхваща два слоя, които се изучават заедно, но се проектират по различни принципи. Първият е преносната среда със своите разновидности: стационарни кабелни мрежи, безжични мрежи с локален обхват, клетъчни мрежи с преминаване между клетки, спътникови връзки с голямо закъснение на разпространение и мрежи с виртуални канали от типа ATM. Вторият е мрежовият слой на стека TCP/IP, който работи над всяка от тези среди и трябва да остава функционален, когато характеристиките им се променят. Учебното съдържание отделя специално внимание на принципите на маршрутизация и управление на потоците в TCP/IP базирани мрежи и именно този слой е предмет на настоящия проект.

Класификацията на протоколите за маршрутизация вътре в една автономна система е установена и се свежда до две семейства. Протоколите по вектор на разстоянието обменят със съседите си собствената си маршрутна таблица, като RIP [1]. Те са прости за реализация, но страдат от бавно разпространение на лошите новини и изискват допълнителни механизми срещу образуването на цикли. Протоколите по състояние на връзките разпространяват описание на локалната свързаност до всички възли, така че всеки възел изгражда еднакъв граф на топологията и изчислява маршрутите самостоятелно. OSPF версия 2 [2] е каноничното описание на този подход, а версия 3 го пренася върху IPv6 [3]. Сравнителен анализ на двете семейства, включително причините за преходните цикли, е даден при Perlman [4].

Изчисляването на маршрутите при протоколите по състояние на връзките се свежда до търсене на дърво от най-кратки пътища с корен текущия възел, тоест до алгоритъма на Дейкстра [5]. Изборът на алгоритъм е безспорен, но времето за сходимост не се определя от него: изчислението е бързо в сравнение с интервалите, през които възлите изобщо научават за промяната. Тези интервали се задават от таймерите за откриване на съседи и от закъснението на лавинното разпространение на обявленията. Затова литературата за ускоряване на сходимостта се съсредоточава върху по-бързо откриване на отказа, например чрез двупосочно откриване на препращането (BFD) [6], а не върху оптимизация на самото изчисление.

Управлението на потоците в TCP/IP се разполага в друг слой, но е свързано със

сходимостта по наблюдаем начин. Прозорецът за претоварване на TCP се свива при загуба на пакети [7, 8], а точно преходните цикли при пренастройване на маршрутите произвеждат загуба, която за подателя е неразличима от загуба поради претоварване. Реакцията на транспортния слой при промяна в маршрутизацията е следствие от поведението на мрежовия слой и това прави времето за сходимост величина с пряк ефект върху пропускателната способност, а не вътрешна характеристика на протокола. Общата рамка на двата слоя е изложена при Kurose и Ross [9], а нормативното описание на TCP е в [10].

Инструментите за изследване на такива преходи също са установени. Симулаторите с дискретни събития OMNeT++ [11] и ns-3 [12] предоставят зряла среда с модели на среди и протоколи. Те обаче симулират собствени модели на протокола, докато настоящият проект се стреми към друго свойство: една и съща реализация да бъде изпълнима както над симулирано, така и над реално време. **[TODO: да се добави преглед на публикации, измерващи сходимостта на OSPF в реални мрежи, след като бъде избран конкретен набор]**

II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА

2.1. Семейство на протокола

Първият избор е между двете семейства протоколи за вътрешна маршрутизация. Протокол по вектор на разстоянието би бил по-кратък за реализация, но при него всеки възел вижда само обобщени разстояния и няма представа за топологията. Измерването на преходни цикли тогава изисква външен наблюдател, който събира таблиците на всички възли и възстановява глобалната картина. При протокол по състояние на връзките всеки възел вече притежава описание на топологията, така че разминаването между представите на два възела е непосредствено наблюдаема величина. Избран е подходът по състояние на връзките именно защото прави измерваната величина видима отвътре.

2.2. Модел на времето

Вторият избор е между три модела на времето. Работа изцяло върху реални интерфейси дава най-висока достоверност, но не дава повтораемост: моментът на отказа не се възпроизвежда и разсейването на резултатите скрива ефекта, който се изследва. Работа изцяло в симулатор дава пълна повтораемост, но проверява модела, а не програмата, която после ще работи в мрежа. Приет е трети вариант: протоколната логика не получава достъп до часовник и до сокети, а взаимодейства с абстракция, зад която стои или дискретно събитийна опашка с виртуален часовник, или системният часовник и реалните сокети. Едни и същи изходни файлове на протокола се компилират в двата режима. Цената на този избор е допълнителна абстракция и невъзможност да се използва пряко системно извикване в протоколния код. Ползата е, че измереното поведение принадлежи на реализацията, а не на нейн симулационен двойник.

2.3. Език и инструментариум за изграждане

Разгледани са C++, Rust и Go. Go предлага най-кратък път до работеща мрежова програма, но неговата среда за изпълнение управлява планирането и събирането на паметта, което внася закъснения, невъзможни за отстраняване от измерването. Rust дава силни гаранции за безопасност на паметта, но еталонните реализации, спрямо които се

сверява коректността, са на C и интеграцията с тях изисква допълнителен слой. Избран е C++ по стандарта от 2020 година [13]: детерминирано управление на ресурсите, пряка съвместимост с интерфейсите на операционната система и наличие на конструкции, които правят абстракцията върху времето безплатна по време на изпълнение. За изграждане е избран CMake, защото трябва да произвежда еднакъв резултат върху Windows, Linux и macOS.

2.4. Еталон за сверяване на коректността

Като еталон са разгледани FRRouting и BIRD. И двата са зрели реализации на OSPF, които работят срещу оборудване на производители и следователно тълкуват [2] по начин, приет от практиката. Еталонът се използва за проверка на коректността, тоест за съвместимост на форматите на пакетите и на съдържанието на изчислените маршрути, а не за сравнение по бързина. Сравнение по бързина би било подвеждащо, защото обхватът на реализацията в този проект е подмножество от този на еталона. **[TODO: да се фиксира конкретната реализация и версия на еталона след проверка кой от двата се пуска по-лесно в контейнер върху всички три платформи]**

2.5. Обхват спрямо учебното съдържание

Дисциплината обхваща и безжични, клетъчни, спътникови и АТМ мрежи. Те са включени в раздел I като контекст, но не в реализацията. Моделирането на канал със загуби и променливо закъснение изисква обосноваване на самия модел и измерената сходимост тогава зависи от достоверността на този модел. Проектът избира по-тесен обхват с ясно измерима величина пред по-широк обхват с недоказуеми числа.

III. ПРОЕКТИРАНЕ НА СИСТЕМАТА

3.1. Обща структура

Системата е разделена на четири слоя. Най-долу стои слой за вход и изход, който предоставя две операции: изпращане на съобщение до съсед и получаване на съобщение. Над него стои източник на време, който предоставя текущ момент и планиране на таймер. Над двата стои протоколното ядро, което съдържа автоматите за състоянията на съседите, базата с обявленията за състояние на връзките и изчисляването на маршрутите. Най-отгоре стои приложната обвивка, която в единия режим е демон върху операционната система, а в другия сценарий на симулатора. Протоколното ядро не съдържа обръщания към часовника и към сокетите. Това ограничение е причината двата режима да изпълняват един и същ код и трябва да бъде проверявано, а не да се разчита на дисциплина при писане. **[TODO: да се добави фигура на слоевете с легенда и препратка към нея в текста]**

3.2. Абстракция върху времето и входа и изхода

В режим на симулация източникът на време е виртуален часовник, който се движи само когато опашката от събития извади следващото събитие. Между две събития не изтича време, така че изпълнението не зависи от натовареността на машината. Планирането на таймер добавя събитие в опашката, а изпращането на съобщение добавя събитие за получаване след закъснение, определено от модела на връзката. При еднакво начално състояние и еднаква последователност от външни събития изпълнението дава един и същ резултат, което е условието за възпроизводимо измерване.

В режим на демон източникът на време е монотонният часовник на операционната система, а входът и изходът са сокети. Тъй като целевите платформи са Windows, Linux и macOS, слой за вход и изход има отделна реализация за всяка от тях зад общ интерфейс, вместо кодът да се пише срещу интерфейса на една платформа и после да се пренася. **[TODO: да се фиксира механизмът за изчакване на събития по платформи, след измерване дали разликата между наличните механизми изобщо влияе на резултата]**

3.3. Протоколно ядро

Ядрото поддържа три структури. Първата е таблица със съседите, всеки от които се намира в състояние от автомата за съседство и има таймери за поддържане и за отпадане. Втората е база с обявленията за състояние на връзките, индексирани по подател и с поредни номера, които определят кое обявление е по-ново. Третата е маршрутната таблица, получена от изчисляването на дървото от най-кратки пътища върху графа, реконструиран от базата. Разпространението на обявленията е лавинно: получено по-ново обявление се записва и се препраща по всички връзки без тази, по която е дошло.

Разделението между база и маршрутна таблица е съществено за измерването. Моментът, в който възел научава за промяна, и моментът, в който възел започва да препраща по новия път, са различни, а разликата между тях е точно интервалът, в който възникват преходни цикли. Ако двете структури бяха обединени, тази разлика би била ненаблюдаема.

3.4. Записване на събития за анализ

Всяко значимо събитие, тоест промяна на състоянието на съсед, приемане на обявление, изчисляване на маршрути и инсталиране на маршрут, се записва с виртуалния или реалния момент на настъпването си в поток от събития. Анализът след изпълнението работи само върху този поток и не се нуждае от достъп до вътрешното състояние. Така една и съща процедура за анализ обработва изхода и от двата режима. **[TODO: да се фиксира форматът на записа на събитията и да се приложи като приложение]**

IV. ПРОГРАМНА РЕАЛИЗАЦИЯ

4.1. Организация на изходния текст

Изходният текст е разделен на библиотека с протоколното ядро, библиотека със слоя за вход, изход и време, и две изпълними програми, които ги свързват: демонът и симулаторът. Разделението е принудително, а не по вкус: ако ядрото е отделна библиотека без зависимост към сокети, нарушение на границата се проявява като грешка при свързване, а не като незабелязана зависимост. Изграждането е с CMake, а тестовите използват GoogleTest.

4.2. Автомат за състоянията на съседите

Съседството преминава през последователност от състояния, в която двата възела установяват двупосочна видимост и след това синхронизират базите си, преди да обявят връзката за използваема [2]. Реализацията описва автомата като явна таблица от преходи, вместо като вложени условни оператори, за да може всеки преход да бъде покрит с отделен тест. **[TODO: да се приложи таблицата на преходите след реализацията и да се посочи покритието]**

4.3. Разпространение на обявленията

Обявлението за състояние на връзка носи подател, пореден номер, възраст и списък от съседи с цени на връзките. Приемането сравнява поредния номер с този на записаното обявление от същия подател, записва по-новото и препраща. Проверката за по-нов номер е критична: без нея лавинното разпространение не спира. **[TODO: да се опише поведението при обръщане на поредния номер и при остаряло обявление]**

4.4. Изчисляване и инсталиране на маршрути

Изчисляването е реализация на алгоритъма на Дийкстра [5] върху графа, реконструиран от базата, с опашка с приоритет. Резултатът е следваща стъпка за всяка достижима цел. Инсталирането е отделна стъпка, за да остане измерим интервалът между изчисляването и влизането на маршрута в сила. **[TODO: да се опише как маршрутът се инсталира в маршрутната таблица на операционната система в режим на демон и**

какви права изисква това на всяка от трите платформи]

4.5. Съвместимост с еталонната реализация

Съвместимостта на формата на пакетите се проверява чрез обмен с еталонна реализация в изолирана среда. Проверява се, че еталонът установява съседство с демона и че изчислените от двете страни маршрути съвпадат за една и съща топология. **[TODO: да се опише средата за проверката и да се приложи резултатът]**

V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА

5.1. Измервани величини

Измерват се две величини. Първата е времето за сходимост, дефинирано като интервалът от момента на отказа на връзката до момента, в който всички възли, които остават свързани, притежават маршрутни таблици, съответстващи на новата топология. Втората е наличието и продължителността на преходни цикли, тоест интервали, в които следващите стъпки на два или повече възела образуват затворен път. Двете величини се извличат от потока от събития след изпълнението и не изискват достъп до вътрешното състояние по време на изпълнение.

5.2. Топологии и сценарии

Експериментът се провежда върху набор от топологии с нарастващ диаметър и нарастваща свързаност, защото и двете величини се очаква да зависят точно от тези два параметъра. Сценарият е един и същ за всяка топология: мрежата достига установено състояние, отказва се една връзка, изпълнението продължава до ново установено състояние. **[TODO: да се фиксира наборът от топологии: брой възли, диаметър, степен на свързаност] [TODO: да се фиксира кои връзки отказват във всяка топология и на какво основание са избрани]**

5.3. Управлявани параметри

Управляваните параметри са интервалът за поддържане на съседството, интервалът за обявяване на отпадане, закъснението на връзките и началното зърно на генератора на псевдослучайни числа. Всяко изпълнение записва пълния набор от параметри заедно с резултата, за да бъде повторимо. **[TODO: да се фиксират стойностите и обхватите на управляваните параметри]**

5.4. Условия за валидност

Резултат се приема за валиден само ако са изпълнени следните условия. Изпълнение с едни и същи параметри и едно и също зърно дава идентичен поток от събития. Мрежата достига установено състояние преди отказа и след него. Крайните маршрутни

таблици съвпадат с изчислените независимо върху пълния граф. Изпълнение, което не удовлетворява някое от условията, се отчита като невалидно и не влиза в резултатите.

5.5. Форма на резултатите

Резултатите се представят в две таблици и една фигура. Първата таблица дава времето за сходимост по топология и по стойност на интервала за отпадане. Втората дава броя и продължителността на преходните цикли за същите комбинации. Фигурата представя времето за сходимост като функция от диаметъра на топологията, с легенда по стойност на интервала за отпадане. **[TODO: броят на повторенията за всяка комбинация се фиксира след пробно изпълнение]**

VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

6.1. Проведени измервания

Измерванията са проведени по методиката от раздел V. Отчетени са изпълненията, удовлетворяващи условията за валидност, а отхвърлените са изброени заедно с причината за отхвърлянето. **[TODO: брой проведени, брой валидни и брой отхвърлени изпълнения, с причините]**

6.2. Време за сходимост

Времето за сходимост по топология и по стойност на интервала за отпадане е дадено в табличен вид (Табл. 1).

Таблица 1. Време за сходимост след отказ на връзка

Топология	Диаметър	Интервал за отпадане	Време за сходимост
[TODO:]	[TODO:]	[TODO:]	[TODO:]
[TODO:]	[TODO:]	[TODO:]	[TODO:]

[TODO: коментар на таблицата: зависи ли времето за сходимост от диаметъра и по какъв закон, и каква част от него се пада на откриването на отказа]

6.3. Преходни цикли

Броят и продължителността на преходните цикли за същите комбинации са дадени (Табл. 2).

Таблица 2. Преходни цикли при пренастройване на маршрутите

Топология	Брой цикли	Средна продължителност	Максимална продължителност
[TODO:]	[TODO:]	[TODO:]	[TODO:]
[TODO:]	[TODO:]	[TODO:]	[TODO:]

[TODO: коментар на таблицата: при кои топологии изобщо възникват цикли и дали продължителността им следва времето за сходимост]

6.4. Сверяване с еталонната реализация

[TODO: резултат от проверката за съвместимост с еталонната реализация: установява ли се съседство, съвпадат ли изчислените маршрути, и къде има разминаване]

6.5. Обсъждане

[TODO: обсъждане на резултатите след попълване на таблиците: кои очаквания се потвърждават, кои не, и какви са ограниченията на измерването]

ЗАКЛЮЧЕНИЕ

Курсовият проект разглежда поведението на мрежа по стека TCP/IP в интервала между две установени състояния на маршрутизацията. Изградени са демон за маршрутизация по състояние на връзките и дискретно-събитиеен симулатор, който изпълнява същата протоколна логика върху управлявано време, така че отказът на връзка да бъде възпроизводимо събитие, а не еднократно наблюдение.

Основната проектна особеност на решението е, че протоколното ядро не притежава достъп до часовника и до сокетите, а работи над абстракция. Това позволява едни и същи изходни файлове да бъдат изпълнявани както в симулация, така и над реални интерфейси, и прави измереното поведение свойство на реализацията, а не на неин отделен симулационен модел. Цената на решението е допълнителна граница между слоевете, която трябва да бъде поддържана и проверявана.

[TODO: обобщение на получените резултати и тяхната инженерна трактовка, попълва се след раздел VI]

[TODO: предимства и недостатъци на решението, оценени спрямо измереното, а не спрямо очакваното]

Като посоки за по-нататъшна работа се очертават разширяване на набора от топологии, включване на механизъм за по-бързо откриване на отказ като този в [6], и свързване на измерената сходимост с наблюдавана пропускателна способност на транспортния слой, което би позволило ефектът от преходните цикли да бъде оценен и откъм приложението.

ЛІТЕРАТУРА

- [1] C. Hedrick, “Routing information protocol,” RFC 1058, Internet Engineering Task Force, June 1988.
- [2] J. Moy, “OSPF version 2,” RFC 2328, Internet Engineering Task Force, April 1998. STD 54.
- [3] R. Coltun, D. Ferguson, J. Moy, and A. Lindem, “OSPF for IPv6,” RFC 5340, Internet Engineering Task Force, July 2008.
- [4] R. Perlman, *Interconnections: Bridges, Routers, Switches, and Internetworking Protocols*. Addison-Wesley, 2 ed., 2000.
- [5] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [6] D. Katz and D. Ward, “Bidirectional forwarding detection (BFD),” RFC 5880, Internet Engineering Task Force, June 2010.
- [7] V. Jacobson, “Congestion avoidance and control,” *ACM SIGCOMM Computer Communication Review*, vol. 18, no. 4, pp. 314–329, 1988.
- [8] M. Allman, V. Paxson, and E. Blanton, “TCP congestion control,” RFC 5681, Internet Engineering Task Force, September 2009.
- [9] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*. Pearson, 8 ed., 2021.
- [10] W. Eddy, “Transmission control protocol (TCP),” RFC 9293, Internet Engineering Task Force, August 2022. STD 7.
- [11] A. Varga and R. Hornig, “An overview of the OMNeT++ simulation environment,” in *Proceedings of the 1st International Conference on Simulation Tools and Techniques for Communications, Networks and Systems (SIMUTools)*, 2008.
- [12] G. F. Riley and T. R. Henderson, “The ns-3 network simulator,” in *Modeling and Tools for Network Simulation*, pp. 15–34, Springer, 2010.
- [13] ISO/IEC, “ISO/IEC 14882:2020 — programming languages — C++,” 2020.

ПРИЛОЖЕНИЯ

Приложение А. Формати на структурите от данни

[TODO: двоичен формат на пакета Hello и на обявлението за състояние на връзка, с полета, размери и подредба на байтовете]

[TODO: формат на записа в потока от събития, използван от процедурата за анализ]

Приложение Б. Описание на топологиите

[TODO: описание на всяка използвана топология: списък на възлите, връзките и цените, във формата, който симулаторът чете]

Приложение В. Изходен текст

[TODO: изходен текст на автомата за състоянията на съседите и на изчисляването на най-кратките пътища, с коментар]

Приложение Г. Пълни резултати от измерванията

[TODO: пълните резултати от всички валидни изпълнения, включително отхвърлените с посочена причина]